

Programmazione Web e Mobile con Laboratorio

Lezione 12 - Node.js 1
(Introduzione, NPM, modulo http)

Node.js

- È un ambiente di esecuzione che permette di eseguire JavaScript sul lato server
- Consente l'esecuzione di codice asincrono e basato su eventi
- È costruito sull'engine open source di Google ad alte prestazioni JavaScript V8, scritto in C++



Cenni Storici

- Creato da Ryan Dahl nel 2009
- Evoluto rapidamente grazie alla sua natura open-source e al vasto supporto della comunità
- Adozione da parte di grandi aziende come Netflix, Uber, e PayPal



Node.js vs JavaScript

Node.js nel server

Ambiente lato server che permette l'esecuzione di JavaScript al di fuori dei browser

Ha accesso a API specifiche del sistema operativo, come quelle per file system, reti e processi

Usato per applicazioni server-side, gestione di API, e operazioni I/O intensive

JavaScript nel client

Viene eseguito come linguaggio di scripting lato client

Accede principalmente a API web come DOM e Web Storage

Focalizzato su interazioni dinamiche all'interno delle pagine web, manipolazione del DOM e comunicazioni asincrone

Node Package Manager (NPM)

- È il gestore di pacchetti ufficiale per Node.js
- Con NPM, è possibile installare i moduli necessari per un progetto con un singolo comando
- Facilita l'installazione, l'aggiornamento e la gestione di librerie e moduli in un progetto
- Utilizza un vasto registro online che contiene migliaia di pacchetti disponibili gratuitamente

Installazione su Linux

- Usa il package manager

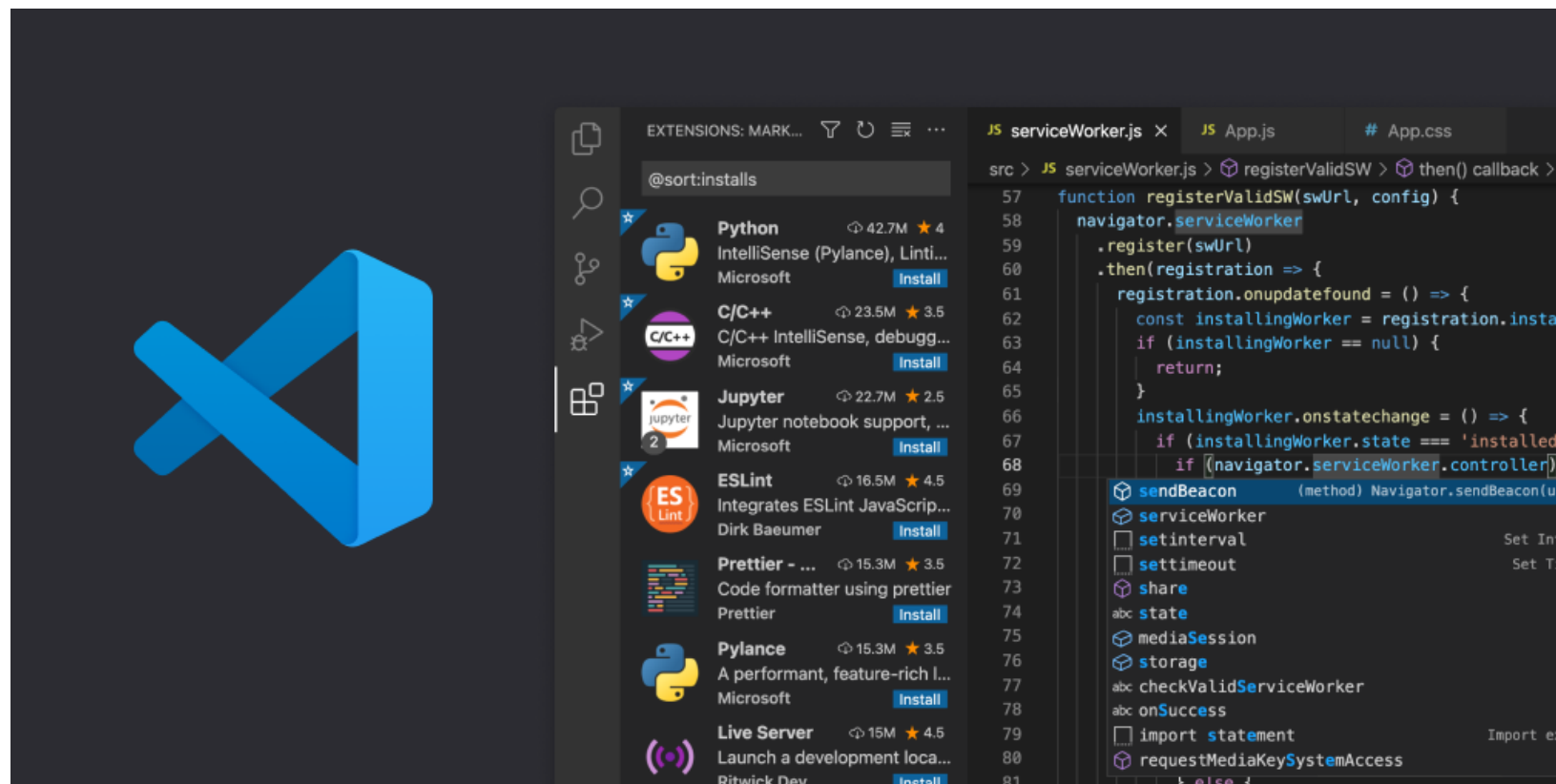
```
sudo apt update  
sudo apt install nodejs npm
```

- Verifica l'installazione

```
node -v  
npm -v
```

Ambiente di Sviluppo

1. IDE Consigliato: Visual Studio Code (VS Code)



7. Alternative: WebStorm, Atom, Sublime Text, ...

Primo Script Node.js

- Creare un nuovo file con estensione .js (es: app.js)
- Scrivere un semplice script, ad esempio

```
console.log("Ciao, mondo!");
```

- Per eseguire lo script:
 - aprire il terminale
 - navigare alla directory che contiene il file app.js
 - eseguire il comando `node app.js`

```
node app.js
```


Esercizio 1

- Creare uno script che genera un numero casuale tra 1 e 10 e lo stampa sulla console

Esercizio 1

- Creare uno script che genera un numero casuale tra 1 e 10 e lo stampa sulla console

```
// Funzione per generare un numero casuale tra 1 e 10
function generateRandomNumber() {
    return Math.floor(Math.random() * 10) + 1;
}

// Chiama la funzione e stampa il risultato
let randomNumber = generateRandomNumber();
console.log("Il numero casuale generato è: " + randomNumber);
```

Esercizio 2

- Creare uno script che genera un numero casuale tra 1 e 10 e poi chiede all'utente di indovinarlo. Se l'utente non indovina correttamente, lo script dà un suggerimento e continua a chiedere finché l'utente non indovina

Esercizio 2

- Creare uno script che genera un numero casuale tra 1 e 10 e poi chiede all'utente di indovinarlo. Se l'utente non indovina correttamente, lo script dà un suggerimento e continua a chiedere finché l'utente non indovina

```
// Genera un numero casuale tra 1 e 10
const randomNumber = Math.floor(Math.random() * 10) + 1;

console.log("Indovina il numero tra 1 e 10.");

// Processa l'input dell'utente da stdin
process.stdin.on("data", function (input) {
  const guess = parseInt(input);
  // Controlla se il numero indovinato è corretto
  if (guess === randomNumber) {
    console.log("Complimenti! Hai indovinato il numero!");
    process.exit(); // Termina il processo
  } else if (guess < randomNumber) {
    console.log("Troppo basso! Prova ancora.");
  } else if (guess > randomNumber) {
    console.log("Troppo alto! Prova ancora.");
  } else {
    console.log("Per favore inserisci un numero valido.");
  }
});
```

Analizziamo il frammento

```
process.stdin.on('data', function (input) {  
  ...  
});
```

- **process** è un oggetto globale in Node.js che gestisce il processo corrente
- **stdin** è uno dei flussi predefiniti accessibili tramite **process** (input utente da tastiera)
- **on()** è un metodo usato per ascoltare eventi su un oggetto. In questo caso, stiamo ascoltando l'evento **data** su **process.stdin**
- L'evento **data** viene emesso ogni volta che l'utente inserisce dati e preme invio
- Quando ciò accade, i dati inseriti dall'utente vengono passati come argomento alla funzione di callback **function (input) {...}**

Moduli

- Un **modulo** è un blocco di codice isolabile che serve a:
 - mantenere il codice organizzato
 - riutilizzare il codice in diverse parti del programma senza riscriverlo
 - incapsulare la logica e i dati, mantenendo il codice privato e protetto da interferenze esterne
- Node.js supporta sia moduli nativi (es. fs, http) sia moduli personalizzati creati dall'utente

Pacchetti

- Un **pacchetto** è una directory che contiene uno o più moduli insieme a un file **package.json** che definisce vari metadati, come dipendenze, script, versione
- I pacchetti sono usati per distribuire e installare moduli e possono essere installati in un progetto Node.js tramite **npm**
- I pacchetti possono essere caricati e utilizzati tramite la funzione **require()**



NPM - Installare Pacchetti

- Installazione **locale** — `npm install <nome_modulo>`
 - Scarica il modulo nella directory corrente da cui si esegue il comando
 - Utilizzabile SOLO dalla directory del progetto via `require( )`
 - Servono da “librerie” (Express, Mongoose, ...)
- Installazione **globale** — `npm install --g <nome_modulo>`
 - Scarica il modulo all'interno di una directory di sistema
 - Utilizzabile da QUALSIASI directory, ma SOLO da console
 - Servono come “strumenti” (Nodemon, Vite, ...)

NPM - Funzionalità

- elenco dei pacchetti installati — `npm list`
- aggiornare un pacchetto — `npm update <nome_modulo>`
- rimuovere un pacchetto — `npm uninstall <nome_modulo>`
- verifica della sicurezza dei pacchetti — `npm audit`
- pulizia delle dipendenze non utilizzate — `npm prune`

Package.json

Gestisce le dipendenze di un progetto, ad esempio:

```
{
  "name": "my-project",
  "version": "1.0.0",
  "description": "A simple Node.js project",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Your Name",
  "license": "MIT",
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.7"
  }
}
```

Package.json

- Elenca le dipendenze e consente l'installazione con `npm install`
- Specifica le versioni dei pacchetti
- Contiene script personalizzati per avviare il server, eseguire test, ecc
- Fornisce metadati sul progetto come nome, versione, autore, ecc
- Facilita la condivisione del progetto con altri sviluppatori
- È necessario per la pubblicazione di pacchetti su npm

Package.json

- Viene creato e aggiornato automaticamente da **npm install**
- Creazione manuale di un file package.json:
 - `npm init`
 - `npm init -y`

Creare un Modulo

1. Creare una nuova directory per modulo, es: “mio-modulo”
2. Dentro la directory del modulo, eseguire il comando **npm init**
3. Creare il file principale del modulo, solitamente: “index.js”, es:

```
function saluta(nome) {  
  return "Ciao, " + nome + "!";  
}  
  
module.exports = { saluta };
```

4. Utilizzare il modulo

```
const mioModulo = require("./mio-modulo");  
  
console.log(mioModulo.saluta('Giorgio'));
```

Modulo http

- È un modulo core (preinstallato)
- Fornisce funzionalità per la gestione di server e client HTTP
- Utile per costruire applicazioni web e API RESTful
- Il metodo **createServer**:
 - Consente di creare un server che può rispondere a richieste HTTP
 - Fornisce metodi per gestire le richieste HTTP in entrata (**req**)
 - Impostare le intestazioni e il corpo delle risposte HTTP (**res**)

Classi di codici HTTP

- **1xx - Informativi**

- 100 Continue: il server ha ricevuto la richiesta e il client può continuare
- 101 Switching Protocols: cambio di protocollo richiesto dal client

- **2xx - Successo**

- 200 OK: la richiesta è andata a buon fine
- 201 Created: risorsa creata con successo

- **3xx - Reindirizzamenti**

- 301 Moved Permanently: la risorsa è stata spostata in modo permanente
- 304 Not Modified: il contenuto non è cambiato (usato per la cache)

- **4xx - Errori del client**

- 400 Bad Request: richiesta errata o malformata
- 401 Unauthorized: accesso negato, autenticazione richiesta
- 403 Forbidden: accesso vietato, anche con autenticazione
- 404 Not Found: risorsa non trovata

- **5xx - Errori del server**

- 500 Internal Server Error: errore generico del server
- 502 Bad Gateway: il server ha ricevuto una risposta errata da un altro server
- 504 Gateway Timeout: il server non ha ricevuto risposta in tempo da un altro server

Esercizio 3

- Creare un server HTTP che risponde con messaggi di testo a diverse richieste GET su percorsi differenti
- Il server deve ascoltare su una porta specifica (ad esempio, 3000)
- Il server deve rispondere a tre percorsi:
 - `/`: risponde con “Benvenuto nella home page”
 - `/time`: restituisce l’ora corrente
 - `/contact`: restituisce un’indirizzo email
- Se viene richiesta una pagina non presente, il server deve rispondere con “404 - Risorsa non trovata”


```
// app.js
const http = require('http');

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

  if (req.url === '/') {
    console.log(req.method);
    res.end('Benvenuto nella home page');
  }
  else if (req.url === '/time') {
    res.end(`Ora corrente: ${new Date().toLocaleTimeString()}`);
  }
  else if (req.url === '/contact') {
    res.end('Contatto: email@example.com');
  }
  else {
    res.statusCode = 404;
    res.end('404 - Risorsa non trovata');
  }
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

Avviare il server

- Opzione 1: lanciare il comando `node app.js`
- Opzione 2: lanciare il comando `npm start`
- In questo caso, assicurarsi che nella sezione `scripts` del file `package.json` sia presente il comando che esegue lo script di avvio, es:

```
{  
  ...,  
  "scripts": {  
    "start": "node app.js"  
  },  
  ...  
}
```

Riavvio automatico

- Installare il pacchetto **nodemon**

```
npm install --save-dev nodemon
```

- Aggiorna package.json

```
{  
  ...,  
  "scripts": {  
    "start": "node index.js"  
    "dev": "nodemon index.js"  
  },  
  ...  
}
```

- Avviare il progetto con il comando `npm run dev`

Modulo fs (File System)

- Serve per interagire con il file system del computer per leggere, scrivere, modificare ed eliminare file e directory
- È un modulo core, quindi non serve installarlo
- Può operare in modo sincrono (bloccante) o asincrono (non bloccante)

Lettura di un file

- Sincrona: blocca l'esecuzione fino a quando il file è letto

```
const data = fs.readFileSync('file.txt', 'utf8');  
console.log(data);
```

- Asincrona: non blocca l'esecuzione e usa una callback

```
fs.readFile('file.txt', 'utf8', (err, data) => {  
  if (err) {  
    console.error('Errore nella lettura:', err);  
    return;  
  }  
  console.log(data);  
});
```

Scrittura di un file

- Sincrona: blocca l'esecuzione fino a quando il file è scritto

```
fs.writeFileSync('file.txt', 'Ciao, mondo!');
```

- Asincrona: non blocca l'esecuzione e usa una callback

```
fs.writeFile('file.txt', 'Ciao, mondo!', err => {  
  if (err) {  
    console.error('Errore nella scrittura:', err);  
  } else {  
    console.log('Scrittura completata!');  
  }  
});
```

Altri metodi

- Append: aggiungere contenuto senza sovrascrivere

```
fs.appendFile('file.txt', '\nNuova riga!', err => {  
  if (err) throw err;  
  console.log('Testo aggiunto!');  
});
```

- Creare/eliminare una cartella

```
fs.mkdirSync('nuova_cartella'); // Sincrono  
fs.mkdir('nuova_cartella', err => { if (err) console.error(err); }); // Asincrono  
  
fs.rmdirSync('nuova_cartella'); // Sincrono  
fs.rmdir('nuova_cartella', err => { if (err) console.error(err); }); // Asincrono
```


Esercizio 4

- Creare un server HTTP in Node.js che risponda con pagine HTML diverse in base al percorso richiesto
 - `/`: index.html
 - `/about`: about.html
 - `/contact`: contact.html
 - **Qualsiasi altro percorso**: 404.html
- Il server deve ascoltare sulla porta 3000
- I file HTML devono essere letti da file esterni


```
const http = require('http');
const fs = require('fs');

const server = http.createServer((req, res) => {
  let filePath = '';

  // Mappa dei percorsi
  if (req.url === '/') {
    filePath = 'index.html';
  } else if (req.url === '/about') {
    filePath = 'about.html';
  } else if (req.url === '/contact') {
    filePath = 'contact.html';
  } else {
    filePath = '404.html';
  }

  // Leggi il file e restituiscilo come risposta
  fs.readFile(filePath, (err, content) => {
    if (err) {
      res.writeHead(500, {'Content-Type': 'text/plain'});
      res.end('Errore interno del server');
    } else {
      res.writeHead(200, {'Content-Type': 'text/html'});
      res.end(content);
    }
  });
});

// Avvia il server sulla porta 3000
server.listen(3000, () => {
  console.log('Server in ascolto su http://localhost:3000');
});
```